WEEK 1

Course Introduction

NLP Applications in Research and Industry

- Me
- Syllabus

- Who are you, and what is your current research "home" (or the area of AI/CS that currently excites you most)?
- What are 2–3 hobbies or deep interests that have nothing to do with computer science?
- What is one specific "unknown" about NLP that you are hoping to demystify by the end of this semester?
- What's the coolest place you've been to, or your favorite food?

Who is this course for?

- Prior NLP experience helpful but not required
  - Can follow Intro to NLP — but designed to also stand alone
  - If you've taken intro: this goes deeper on specific applications
  - If you haven't: foundational concepts will be introduced as needed
- What you'll bring
  - Comfort with Python and basic ML concepts
  - Willingness to engage with both code and critical thinking
  - Curiosity about where NLP actually gets deployed
- What this course is NOT
  - Not a broad survey of NLP methods
  - Not a theory-first course — we build things
  - Not a course where you just call APIs and submit outputs

What makes this course different

Application-first

Every technique is taught in context of a real deployment scenario. We don't do tokenization in a vacuum — we do it because a medical QA system needs it.

Depth over breadth

Four applications across 14 weeks. We go deep on each: architecture, evaluation, reliability, privacy, fairness, deployment.

Industry realism

The 12-step framework mirrors how real NLP systems get built. You will think about what happens after the model runs.

Code as craft

Best practices matter. Submissions are GitHub repos. Code is evaluated, not just 'does it run.'

The Four Applications

What we'll build this semester

Four application blocks

1 · Medical QA (Weeks 2–5)

Build a question-answering system over clinical text. Introduces RAG, hallucination, privacy, and evaluation fundamentals.

2 · Legal Document Retrieval (Weeks 6–8)

Retrieve and summarize legal documents. Goes deep on retrieval systems, chunking, hybrid search, and summarization faithfulness.

3 · Educational Tutoring (Weeks 9–11)

Design an NLP-powered tutoring system. Covers prompting, LLM-as-judge, fairness, interpretability, and human-in-the-loop.

4 · General-Purpose Assistants (Weeks 12–14)

Scale up: MoE architectures, alignment, safety, deployment infrastructure, and monitoring in production.

Grading & Expectations

How your work will be evaluated

Components

CODING + ANALYSIS

- Weekly Coding Assignments
  - One per week, submitted as a GitHub repo
  - Scoped to a concept from the previous lecture
  - Graded on correctness, code quality, and depth
  - Minimal vs. thorough is explicitly specified
- Research Paper Analysis
  - One per application block (four total)
  - 2–3 papers given; write a critical analysis
  - Not a summary — assume the reader has read them
  - Engage with design choices, results, and what you'd do differently

PIPELINE PROJECTS

- Application Pipeline Projects
  - One per application block, due one week after block ends
  - Two components: Report + Code
- Report component
  - 12-step framework: Problem → Prior Work → Data → Constraints → Architecture → Evaluation → Reliability → Privacy → Fairness → Interpretability → Deployment → Impact
  - Every step must be addressed — depth over length
- Code component
  - Full end-to-end pipeline
  - 2–3 datasets, evaluation, training or inference
  - Submitted as a GitHub repository

The 12-step project framework

- Why a framework?
  - Most ML courses stop at 'does the model perform well on the test set'
  - Real systems fail for reasons that have nothing to do with model accuracy
  - The framework forces you to think through the full lifecycle before you code
- The 12 steps
  - Problem Definition · Prior Work · Data · Constraints
  - Architecture · Evaluation · Reliability & Safety · Privacy & Legal
  - Fairness & Bias · Interpretability · Deployment · Impact
- How it's used in this course
  - Each application block ends with a project applying all 12 steps to what we built
  - Your project report must address every step — some briefly, some deeply

Reading papers critically

- What a critical analysis IS
  - Engaging with what the authors were trying to do and why
  - Evaluating their design choices: models, datasets, evaluation metrics
  - Asking whether results support what is claimed
  - Identifying what you would do differently and why
  - Drawing connections across the provided papers and to course material
- What a critical analysis is NOT
  - A summary of what each section of the paper says
  - Praise without justification ('the paper makes a strong contribution')
  - A list of limitations with no engagement
- The goal
  - Develop the habit of reading research skeptically — published results are not ground truth
  - You'll be a better practitioner if you can identify when a benchmark doesn't measure what it claims

What is NLP?

And why does it matter for applications?

Natural Language Processing — a working definition

- NLP is the field at the intersection of linguistics, computer science, and ML
  - Goal: enable computers to understand, generate, and reason about human language
  - Language is the medium humans use to record knowledge — NLP is how machines access that knowledge
- The core challenge: language is ambiguous
  - "I saw the man with the telescope" — who has the telescope?
  - "She fed her cat tuna" — whose cat? what kind of tuna?
  - Meaning depends on context, world knowledge, shared assumptions
- Why applications?
  - The techniques only make sense in context of a problem
  - A tokenizer is trivial — a tokenizer that breaks on clinical abbreviations is not
  - Every design choice in this course has a reason rooted in the application

APPLICATIONS

Where NLP is deployed

Healthcare & Biomedicine

Clinical note analysis, medical QA, drug interaction extraction, radiology report summarization

Legal & Compliance

Contract review, case law retrieval, regulatory document analysis, GDPR compliance scanning

Education & Tutoring

Automated feedback, essay grading, personalized tutoring, reading comprehension assessment

General Assistants

Chatbots, coding assistants, search, summarization, translation — the applications behind daily-use AI products

Why these four applications specifically

- They span the space of NLP deployment challenges
  - Medical QA: high-stakes correctness, privacy constraints, specialized language
  - Legal retrieval: precision vs recall, long documents, catastrophic hallucination costs
  - Educational tutoring: subjective evaluation, fairness across populations
  - General assistants: scale, alignment, cost, serving millions of diverse users
- They build on each other deliberately
  - RAG introduced in Medical QA reappears in Legal retrieval
  - Hallucination first seen in week 3 becomes more nuanced by week 8
  - Evaluation methods compound: EM → faithfulness metrics → LLM-as-judge → retrieval metrics
- They represent high societal impact
  - These are not toy domains — mistakes in healthcare or legal retrieval cost people

Tools, setup, and course logistics

- Coding environment
  - Python 3.10+, PyTorch, HuggingFace Transformers & Datasets
  - All assignments submitted as GitHub repositories — set yours up today
  - We use open-source models where possible (privacy motivation comes up in App 1)
- Weekly rhythm
  - Lecture: core concepts
  - Assignment released same day, due before the following lecture
- Collaboration policy
  - Discuss ideas freely, write your own code
  - Paper analyses are individual — no collaboration
  - LLM-assisted coding is allowed – though, the more you write yourself the more you'll understand!

TODAY

Week 1 activity — map the problem space

- In groups, pick one of the four application domains
  - You don't need prior knowledge — use intuition and common sense
- Spend 20 minutes answering
  - Who are the users? What do they need from the system?
  - What does a failure look like? How bad is it?
  - What data would you need? Do you think it exists?
  - What would make you trust the system's output?
  - What models can/should we use?
  - Will users have any issues with the models we chose?
  - Does training/evaluation data exist?
  - Are there biases in the data we need to worry about?
  - Can the model hallucinate? How badly does that matter here?
  - How do we monitor the system in production?
- Then we discuss as a class
  - Goal: surface the real complexity before we've seen any techniques

Activity – map the problem space

Healthcare and Biomedicine

- Who are the Users? What do they need from the System?
  - → Users: Physicians, Nurses, Chemists/Biologist in compound / treatment Research, Nurses
  - → Needs: Case-specific, accurate Information, confidentiality, Reliability, understandable
- What does failure look like? How bad is it?
  - → Failure can have many forms → wrong treatment suggestion, false information when Researching, Misunderstanding of Situation for Patient / Drug development
  - → Wrong drugs / treatments can cost a lot of money and in the worst-case lives
- What data would you need? Do you think it exists?
  - → Data: Patientrecords, Research Papers, FDA-Labels, Clinical Trials,
  - → Existence: Yes

- What would make you trust the systems output?
  - → Accordance with the opinions / views from human experts and quantifiable similarity to test data
- What models can/should we use?
  - → ML Models for Classificationtasks, LLM for Reasoning
- Will users have any issues with the models we choose?
  - → As long as the output is relevant, helpful and correct, there should not be to many issues.
- Does training / evaluation data exist?
  - → Strongly depends on context generally speaking yes
- Are there biases in the data that we need to worry about?
  - → Any human data can be biased, eg. can proffessional Physicians can choose wrong / non Sota Treatment
- Can the model hallucinate? How badly does that matter here?
  - → The model can hallucinate and produce severe output in the right context

What's coming — Week 2

- Next lecture (start of Medical QA): QA & Language Models
  - What question answering is as a task — extractive vs. generative
  - How language models work: tokenization, embeddings, attention — enough to debug
  - Decoding strategies — temperature, top-k, top-p — why they matter for generation quality
  - Pretraining: what it gives you and what it doesn't
  - Introduction to supervised fine-tuning
- Before next class
  - Set up your Python environment and GitHub
  - Run a HuggingFace pipeline on any task (question answering, translation, summarization, etc) – just to make sure your code and environment work (see huggingface tutorials on their website – https://huggingface.co/docs/transformers/en/tasks/question_answering)
- Assignment 1 will be released after next lecture
  - Scope: basic QA pipeline using a pretrained model
  - Details provided in lecture 2